RNN Intuition II

Structured Notes

Forward Pass Mathematics, Tensor

Shapes, Weight Sharing,
Unrolling Across Time, and Back Propagation Through Time

Contents

1 RNN Single Step Review **2**
1.1 Why CNN Is Not Enough for Sequential Data . . . 2
1.2 Single Step RNN Operation . . . 2

2 Tensor Shapes in RNN **3**
2.1 Shape Definitions . . . 3
2.2 Matrix Multiplication Walkthrough . . . 3
2.3 Code: Single RNN Step with Shapes . . . 3

3 Hidden State: Memory in RNN **4**
3.1 Why Does RNN Require Previous Hidden State? . . . 4
3.2 Role of the Current Hidden State h_t . . . 4
3.3 Why RNN Works Better Than ANN for Sequential Data . . . 4

4 RNN Unrolling Across Time **4**
4.1 From Single Cell to Sequence Processing . . . 4
4.2 Recursion Analogy . . . 5
4.2.1 Fibonacci Series Example . . . 5
4.2.2 Factorial Example . . . 5
4.3 What Does Unrolling an RNN Mean? . . . 5
4.4 Base Condition h_0 . . . 5

5 Weight Sharing in RNN **6**
5.1 Same Weights Across All Time Steps . . . 6
5.2 Why Weight Sharing? . . . 6
5.3 Parameter Efficiency Comparison . . . 6
5.4 Comparison: ANN vs RNN Weight Structure . . . 7

6 Forward Pass Computation **7**
6.1 Full Forward Pass Through Time . . . 7
6.2 Connection to ARMA/ARIMA Models . . . 7
6.3 Why tanh Activation? . . . 7

7 Back Propagation Through Time (BPTT) **8**
7.1 How RNN Learns . . . 8
7.2 Loss Computation Across Time . . . 8
7.3 Gradient Flow Backward Through Time . . . 8
7.4 Gradient Accumulation for Shared Weights . . . 8
7.5 PyTorch RNN Implementation . . . 9

8 Summary **10**

1 RNN Single Step Review

Recurrent Neural Network

An **RNN** is not just a model—it is a **framework**. It works anywhere there is sequential data. Models use RNN as the underlying architecture to process sequences where order and memory matter.

1.1 Why CNN Is Not Enough for Sequential Data

CNN lacks two critical properties required for sequential data:

- **Memory**: CNN does not retain information from previous inputs.
- **Order**: CNN does not inherently capture the ordering of data points.

RNN addresses both by carrying a **hidden state** forward in time. Unlike ANN and CNN, the hidden state is carried forward, making RNN recurrent (recursive) in nature.

1.2 Single Step RNN Operation

At time step t, the RNN takes:

- **Current input**: x_t
- **Previous memory**: h_{t-1} (hidden state from the previous time step)

It produces:

- **New memory**: h_t (updated hidden state)
- **Output**: y_t

Hidden State Update:

$$h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$$

Output Generation:

$$y_t = W_y \cdot h_t + b_y$$

Where:

- W_x — weight matrix for current input influence
- W_h — weight matrix for past memory influence
- b — bias term
- tanh — non-linearity to control values
- W_y — weight matrix for output generation
- b_y — output bias

Two operations happen at each step:

1. The **updated memory** is produced by combining the new input with the previous memory. This captures the relationship between the current input and all previous inputs.
2. Based on this relationship, the **output** is generated via a simple linear transformation.

Core Mechanism

The updated memory contains the relationship between the current input and the previous memory. Based on this relationship, the output is generated. This is the core mechanism of RNN—it captures how previous data relates to current data.

2 Tensor Shapes in RNN

2.1 Shape Definitions

Assume:

- Batch size: B
- Input size (dimension): D
- Hidden size: H
- Output size: O

Tensor	Shape	Description
x_t	(B, D)	Input at time step t
h_t	(B, H)	Hidden state at time step t
y_t	(B, O)	Output at time step t
W_x	(H, D)	Input-to-hidden weight matrix
W_h	(H, H)	Hidden-to-hidden weight matrix
W_y	(O, H)	Hidden-to-output weight matrix

Batch Size Is Constant

The batch size B remains the same throughout the entire computation. Once data enters the network with batch size 32, it stays 32 at every step. Only the feature dimensions change as data flows through the network.

2.2 Matrix Multiplication Walkthrough

Step-by-step shape verification for $h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$:

1. $W_x \cdot x_t$: $(H, D) \times (B, D)^T \Rightarrow (B, H)$ — the D dimensions cancel out
2. $W_h \cdot h_{t-1}$: $(H, H) \times (B, H)^T \Rightarrow (B, H)$ — the H dimensions cancel out
3. Bias b: (B, H)
4. All three terms have shape (B, H), so they can be added element-wise
5. Result: h_t has shape (B, H)

Output computation:

1. $W_y \cdot h_t$: $(O, H) \times (B, H)^T \Rightarrow (B, O)$
2. Bias b_y: (B, O)
3. Result: y_t has shape (B, O)

2.3 Code: Single RNN Step with Shapes

Single RNN Step — Shape Verification

```python
import torch

# Define dimensions
B = 2    # batch size
D = 5    # input dimension
H = 4    # hidden size
O = 3    # output size

# Input tensor: (B, D)
x_t = torch.randn(B, D)            # shape: (2, 5)

# Previous hidden state: (B, H)
```

```
h_prev = torch.randn(B, H)          # shape: (2, 4)

# Weight matrices
W_x = torch.randn(H, D)             # shape: (4, 5)
W_h = torch.randn(H, H)             # shape: (4, 4)
W_y = torch.randn(O, H)             # shape: (3, 4)

# Hidden state update: (B, H)
h_t = torch.tanh(x_t @ W_x.T + h_prev @ W_h.T)  # (2, 4)

# Output: (B, O)
y_t = h_t @ W_y.T                                # (2, 3)
```

Listing 1: Verifying tensor shapes in a single RNN step

3 Hidden State: Memory in RNN

3.1 Why Does RNN Require Previous Hidden State?

The previous hidden state h_{t-1} stores information from earlier time steps. This allows RNN to remember past information and use it to influence the current prediction. Without h_{t-1}, each time step would be independent—no different from a feedforward network.

3.2 Role of the Current Hidden State h_t

The current hidden state acts as **memory**. It captures sequential dependency and transfers learned information across time steps. Specifically, the current h_t will act as h_{t-1} for the next input x_{t+1}.

Two Different Questions

"Why does RNN require previous hidden state?" and "What is the role of the current hidden state?" are fundamentally different questions:

- **Previous** h_{t-1}: Carries information from the past into the current computation.
- **Current** h_t: Combines past and present, then passes this combined knowledge to the future.

3.3 Why RNN Works Better Than ANN for Sequential Data

RNN maintains hidden states that allow it to learn **temporal relationships** between data points. ANN processes each input independently with no mechanism to carry information across inputs. The hidden state is what gives RNN its ability to model sequences.

Memory Flow

Memory flow across time is what makes RNN powerful. The model learns sequential patterns where past influences future. This property makes RNN suitable for language, time series, speech, and sequential prediction tasks.

4 RNN Unrolling Across Time

4.1 From Single Cell to Sequence Processing

Sequential data contains multiple time steps. For example, the sentence "I love deep learning" becomes:

- x_1 = "I"
- x_2 = "love"
- x_3 = "deep"

- x_4 = "learning"

Instead of using one RNN cell, the same RNN is **repeated across time**. This is how recursion is applied—the same function is called again and again.

RNN Unrolled Across Time Steps:

$$h_1 = f(x_1, h_0)$$
$$h_2 = f(x_2, h_1)$$
$$h_3 = f(x_3, h_2)$$
$$h_4 = f(x_4, h_3)$$

Where h_0 is the **base condition**—an initial hidden state that must be provided to start the recursion.

4.2 Recursion Analogy

RNN unrolling is directly analogous to recursion in programming. A recursive function requires:

1. A **base condition** to prevent infinite loops
2. A **recursive step** that depends on the previous value

4.2.1 Fibonacci Series Example

Base conditions: $F(1) = 1$, $F(2) = 1$

Recursive definition: $F(n) = F(n-1) + F(n-2)$

Computation: $F(3) = F(2) + F(1) = 2$, $F(4) = F(3) + F(2) = 3$, and so on.

4.2.2 Factorial Example

Base condition: $0! = 1$

Recursive definition: $n! = n \times (n-1)!$

Computation: $5! = 5 \times 4! = 5 \times 4 \times 3! = \ldots$

Long-Range Dependency Problem

Consider computing 5!: when calculating 0! at the deepest recursion level, it has no knowledge that 5! or even 4! ever existed. Similarly, 2! only knows about 3! from its direct caller. In RNN, this is the **long-range dependency problem**—information from distant time steps gets diluted as it passes through many recursive calls.

4.3 What Does Unrolling an RNN Mean?

Unrolling an RNN means **expanding the RNN across time steps** to show how the hidden state flows sequentially from one step to another. It is a visualization and computational technique that makes the temporal dependencies explicit.

4.4 Base Condition h_0

The base condition h_0 depends on the data being processed. It can be:

- A default token value
- A zero vector

- Any initial value appropriate for the task

The hidden state will keep updating from this starting point. The key principle: h_t is a function of x_t and h_{t-1}.

5 Weight Sharing in RNN

5.1 Same Weights Across All Time Steps

Weight Sharing

In RNN, the **same weights** W_x, W_h, and W_y are used at every time step. This is called **weight sharing**. The weights do not change from one time step to the next during the forward pass—only the hidden state is updated.

This happens because the function is recursive. Calling the same function again and again means the same weights are used across all time stamps.

Hidden State vs Weight Updates

Hidden state is updated during the **forward pass**—at each time step, a new h_t is computed.

Weights are updated during **back propagation**—only after the loss is calculated over the entire sequence.

These are two fundamentally different update mechanisms operating at different phases of training.

5.2 Why Weight Sharing?

1. **Sequence length can vary**: If different weights were used for each time step, the model would need separate weight matrices for each possible sequence length. This is impractical.
2. **Model must generalize across time**: A single set of weights that works at every time step ensures the model generalizes well to variable-length sequences.
3. **Fewer parameters required**: Weight sharing dramatically reduces the number of parameters.

If different weights were used at each recursion step, the relation would no longer be a **recurrence relation**—the function itself would keep changing.

5.3 Parameter Efficiency Comparison

Property	Without Sharing	With Sharing
Sequence length 10	10 different weight sets	1 weight set reused
Parameters	10× base count	1× base count
Example	7,000 parameters	700 parameters
Training speed	Slower	Faster
Generalization	Poorer	Better

Quick Calculation: If a model without weight sharing has 7,000 parameters and with weight sharing has 700 parameters, the sequence length is:

$$\text{Sequence length} = \frac{7{,}000}{700} = 10$$

5.4 Comparison: ANN vs RNN Weight Structure

Aspect	ANN	RNN
Structure	Layer-wise, no time steps	Single structure across time
Weights	Separate per layer: $W_1, W_2, W_3, \ldots$	Same reused: W_x, W_h, W_y
Time steps	Not applicable	$T_1, T_2, T_3, \ldots, T_n$
Sequential learning	Not supported	Supported via hidden state

DSA Connection

A simple DSA concept—**recursion**—solved the temporal dependency problem in neural networks. Instead of using different weights across timestamps, the designers used recursion: same function, same weights, called repeatedly. This is more efficient, saves computation time, and achieves the goal of learning temporal patterns.

6 Forward Pass Computation

6.1 Full Forward Pass Through Time

At each time step, RNN performs two operations:

1. **Hidden state update**: $h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$
2. **Output generation**: $y_t = W_y \cdot h_t + b_y$

This repeats across all time steps in the sequence.

Full Sequence Computation for input sequence x_1, x_2, x_3, x_4:

$$
\begin{aligned}
h_1 &= f(x_1, h_0) \quad (h_0 \text{ is the base condition}) \\
h_2 &= f(x_2, h_1) \\
h_3 &= f(x_3, h_2) \\
h_4 &= f(x_4, h_3)
\end{aligned}
$$

Each step depends on the previous step—this is sequential learning.

6.2 Connection to ARMA/ARIMA Models

The RNN forward pass mathematics is similar to ARMA (AutoRegressive Moving Average) models. In ARMA, the current value also depends on previous values. The key difference is that RNN introduces a hidden state layer, but the underlying mathematical principle of temporal dependency is the same. RNN can be thought of as performing auto-regression through its hidden state mechanism.

6.3 Why tanh Activation?

Property	Explanation
Non-linearity	Linear models cannot learn complex relationships
Value control	Keeps output bounded between -1 and $+1$
Negative values	Negative relationships are equally important as positive
Sequential context	If previous relationship is negative, tanh preserves that signal

Vanishing Gradient Problem

The biggest problem with tanh is the **vanishing gradient**. Since tanh outputs values between -1 and $+1$, these small values get multiplied repeatedly during back propagation. Like sigmoid, after 2–3 back propagation steps, the gradient effectively vanishes—there is no gradient left to update the weights. This is a fundamental limitation of vanilla RNN.

7 Back Propagation Through Time (BPTT)

7.1 How RNN Learns

Like ANN, RNN uses:

- Loss function
- Gradient descent
- Back propagation

The critical difference:

- **ANN**: Back propagation across **layers**
- **RNN**: Back propagation through **time** (BPTT)

Back Propagation Through Time (BPTT)

BPTT is the process of computing gradients across multiple time states in an unrolled RNN. Instead of propagating errors backward through layers (as in ANN), errors propagate backward through time steps.

7.2 Loss Computation Across Time

Forward pass produces outputs at each time step:

$$y_1, y_2, y_3, y_4$$

Loss is computed at each time step:

$$L_1, L_2, L_3, L_4$$

Total loss is the sum of all time step losses:

$$L = L_1 + L_2 + L_3 + L_4 = \sum_{t=1}^{T} L_t$$

Optimization is performed on this total loss, not on individual time step losses.

7.3 Gradient Flow Backward Through Time

The gradient flows backward from the total loss through the hidden states:

$$\frac{\partial L}{\partial h_4} \rightarrow \frac{\partial L}{\partial h_3} \rightarrow \frac{\partial L}{\partial h_2} \rightarrow \frac{\partial L}{\partial h_1}$$

This happens because each hidden state depends on the previous hidden state (h_t depends on h_{t-1}), so the gradient **must** flow through all time steps.

7.4 Gradient Accumulation for Shared Weights

Since the same weights are used across all time steps, gradients from all time steps **accumulate**:

$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

This applies to all shared weight matrices: W_x, W_h, and W_y. The gradients from every time step are summed together to produce the final weight update.

Training Cycle

The complete training cycle follows the standard pattern:

1. **Forward pass**: Compute all hidden states and outputs across time
2. **Compute total loss**: Sum losses from all time steps
3. **Back propagation through time**: Compute accumulated gradients
4. **Update weights**: Apply gradient descent to shared weights
5. **Zero gradients**: Reset for the next epoch

Feed forward → loss calculation → back propagation → weight update. This cycle is the same as in ANN/CNN—the only difference is that back propagation happens through **time** instead of through **layers**.

7.5 PyTorch RNN Implementation

RNN Forward Pass and BPTT in PyTorch

```python
import torch
import torch.nn as nn

# Dimensions
T = 4    # sequence length
B = 2    # batch size
D = 5    # input dimension
H = 4    # hidden size

# Create RNN: pass input_size first, then hidden_size
rnn = nn.RNN(D, H)

# Input: (T, B, D) - sequence length first
x = torch.randn(T, B, D)

# Initial hidden state: (1, B, H)
h0 = torch.zeros(1, B, H)

# Forward pass
output, h_n = rnn(x, h0)
# output shape: (T, B, H) - hidden states at all time steps
# h_n shape: (1, B, H) - final hidden state

# Target for regression
target = torch.randn(T, B, H)

# Total loss (mean squared error across all time steps)
loss = ((output - target) ** 2).mean()

# Back propagation through time
```

```
loss.backward()
```

Listing 2: Using torch.nn.RNN for forward pass and loss computation

PyTorch RNN Input Convention

In `torch.nn.RNN`, the constructor takes `(input_size, hidden_size)`—input dimension D first, then hidden dimension H. The input tensor shape is `(T, B, D)`—sequence length first, then batch size, then input dimension. This ordering is important to remember.

8 Summary

- RNN is a **framework** for sequential data, not just a model. It addresses the memory and order limitations of CNN/ANN.
- The **hidden state** h_t is the core mechanism—it combines current input with past memory to produce updated memory and output.
- **Tensor shapes**: Input (B, D), Hidden (B, H), Output (B, O), with weight matrices $W_x(H, D)$, $W_h(H, H)$, $W_y(O, H)$.
- RNN **unrolls across time** by applying the same function recursively: $h_t = f(x_t, h_{t-1})$ with base condition h_0.
- **Weight sharing** uses identical W_x, W_h, W_y at every time step, reducing parameters dramatically (e.g., 7,000 $\rightarrow$ 700 for sequence length 10).
- **tanh** activation provides non-linearity and bounds values to $[-1, +1]$, but suffers from the vanishing gradient problem.
- **BPTT** computes gradients backward through time steps. Total loss $L = \sum L_t$, and gradients accumulate: $\frac{\partial L}{\partial W} = \sum_t \frac{\partial L_t}{\partial W}$.
- Hidden states update during **forward pass**; weights update during **back propagation**—these are distinct mechanisms.